Lecture 2: Variables, Data Types & Operators

DSA Series | Shradha Khapra (Apna College) | C++ | Short Notes

1. Output: cout

- Screen par kuch print karna ho to **cout** use karte hain. Text **double quotes** ke andar likhte hain, wo jaisa hai waisa print hota hai.
- C++ **case sensitive** hai: `cout` ko `Cout` likha to error. C++, Java, Python sab case sensitive hain.
- Har statement ke end mein **semicolon (;)** lagta hai. Isko **statement terminator** kehte hain, ye English ke full stop jaisa hai.

```
cout << "Apna College";
```

2. VS Code mein file banana

- VS Code kholo, **Explorer** icon se **Open Folder** karke apna folder chuno (ya naya banao).
- Folder mein **New File** banao, naam jaise **code.cpp**.
- **.cpp** = C++ file ka extension (text file ka .txt hota hai waise).
- Code **Ctrl+S** (Mac: Cmd+S) se save karo. Red lines = error hai, hover karke dekh sakte ho.

3. Boilerplate code

```
#include <iostream>
using namespace std;

int main() {
    // kaam yahan likho
    return 0;
}
```

- **int main()**: **main function** hai. Flowchart ke **Start** jaisa, code yahin se run hona shuru hota hai. Kaam **{ }** (curly braces) ke andar likhte hain.
- **#include <iostream>**: **preprocessor directive**. Compiler ko batata hai ki iostream file jodni hai, jisme cout ka logic likha hai.
- **using namespace std;**: **std** C++ ka standard namespace hai (ise abhi ek file ki tarah socho). Ye line na likho to har baar `std::cout` likhna padega.
- **return 0;**: main function ek int value return karta hai. Iske bina bhi code chalega, par likhna achhi aadat hai.
- Ye poora code **boilerplate code** kehlata hai, har DSA question mein isi se shuru karte hain. Yaad karne ki zaroorat nahi, aadat ho jayegi.

5. Nayi line aur ek hi line mein print

- Output ke baad nayi line chahiye to **endl** ya **\n** lagao. Na lagao to terminal mein end par **%** jaisa sign aa sakta hai (trailing character).
- **\n** (backslash n) thoda fast hota hai, competitive programming mein **\n** prefer karte hain. Seekhte time dono barabar hain.
- **\n** ke baad jo likhoge wo agli line par print hoga.
- Ek hi line par kai cheezein: beech mein **endl / \n** mat lagao, spaces do.

```
cout << "Hello World" << endl;
cout << "Hello World\n";
cout << "Apna College " << "Hello World" << endl;   // ek hi line
cout << "Hello\nWorld";   // do line
```

Practice: apna first name aur last name **do alag lines** mein, sirf **ek cout statement** se print karo. (Hint: \n)

6. Comments

- `//` ke baad likha hua **comment** hai. Compiler ise **ignore** karta hai, ye **humans ke liye** hota hai.
- Team mein naya developer code samajh sake, isliye comments likhte hain (e.g. `// starting point of execution`). Aam taur par English mein.
- Shortcut: **Ctrl + /** (Mac: **Cmd + /**), dobara dabane par uncomment. Poori file select karke bhi kar sakte ho.

7. Variables

- Math mein a = 5, x = 10 jaise. Variable ek **container** hai jisme data store hota hai (number ya character).
- Computer ki **RAM** mein jagah (box) allocate hoti hai, us par variable ka naam lagta hai aur value store hoti hai.
- **Variable** = jo **vary** kar sake. Value kal badal sakti hai (age 25 se 26), naam wahi rehta hai.
- **Naam ke rules:** underscore _ ya **letter (a-z)** se shuru hona chahiye, **digit se nahi** (9age galat). Variable ke naam ko **identifier** kehte hain.
- **Single character** single quotes mein: `'A'`. **Kai characters (string)** double quotes mein: `"ABCD"`.

8. Data Types

- Variable banate time **type** batana padta hai, taaki compiler jaan sake ki kis tarah ka data store hoga.
- `int age = 25;` (declaration + initialization). Type batane par hi statement complete hoti hai.

Type	Size	Kya store karta hai	Example
int	4 bytes	Whole numbers (+, -, 0)	int age = 25;
char	1 byte	Single character	char grade = 'A';
float	4 bytes	Decimal (value ke baad **f** lagao)	float PI = 3.14f;
bool	1 byte	true / false	bool isSafe = true;
double	8 bytes	Decimal (float se double size)	double price = 100.9;

- Ye 5 **primitive** (sabse basic) data types hain. Non-primitive (arrays, strings) aage padhenge.
- **sizeof():** size bytes mein batata hai. `cout << sizeof(age);` -> 4
- **float ke baad f:** bina f ke compiler decimal ko **double** maanta hai. Decimal literal by default double hota hai.
- **Constant** (jiski value change nahi hoti, jaise PI) ka naam aam taur par **CAPITAL** mein likhte hain.
- **bool print:** true -> **1**, false -> **0** print hota hai.

Memory mein data kaise store hota hai

- Computer ko sirf **0 aur 1** (binary) samajh aata hai. Ek 0/1 = **1 bit**. **8 bits = 1 byte**.
- **int = 4 bytes = 32 bits** reserve hote hain, chahe sab use na ho.
- Number seedha store nahi hota, binary mein convert hota hai. Jaise **25 = 11001** (binary chapter mein seekhenge).
- **char** ke liye pehle uski **ASCII value** nikalti hai, phir wo binary mein store hoti hai. ASCII = American Standard Code for Information Interchange.
- **A = 65, B = 66, C = 67** ... aur **a = 97, b = 98, c = 99** ... Ye har language mein fixed hain.
- Positive aur negative numbers store karne ka tareeka alag hota hai (binary chapter mein).

9. Type Conversion aur Type Casting

	Type Conversion	Type Casting
Kaun karta hai	**Implicit**: compiler automatic	**Explicit**: programmer manually
Kab	Chhote type -> bade type (data loss nahi)	Aam taur par bade -> chhote type
Example	char -> int, float -> double	(int)price

```
char grade = 'A';
int value = grade;        // implicit: 65 (ASCII value)

double price = 100.9;
int newPrice = (int)price; // explicit: 100
```

- Double se int mein casting par **decimal ke baad ka hissa kat jaata hai**, round nahi hota (1.99 bhi 1 banta hai, math ki tarah 2 nahi).

10. Input: cin

- Input lene ke liye **cin >> variable;** (cout ki tarah, arrows ki direction alag).
- Variable bana kar value na do to usme **garbage value** hoti hai (random, kabhi 0 bhi).
- Pehle cout se user ko batao, phir cin se value lo. Jab tak input nahi doge, program aage nahi badhega.

```
int age;
cout << "Enter the age: ";
cin >> age;
cout << "Your age is " << age << endl;
```

- **cin** aur **cout** functions nahi, **global objects** hain; **<< >>** overloaded operators hain (OOP chapter mein samjhenge). Abhi bas yaad rakho.

11. Operators

A. Arithmetic

Operator	Naam	Example (a = 10, b = 5)
+	Sum	a + b = 15
-	Difference	a - b = 5
*	Product	a * b = 50
/	Division	a / b = 2
%	Modulo (remainder)	11 % 5 = 1, 10 % 5 = 0

- **Integer division:** int / int ka answer **int** hota hai, decimal wala hissa kat jaata hai (round nahi). **5 / 2 = 2**, **10 / 3 = 3**.
- Decimal answer chahiye to ek taraf **float ya double** rakho. Result hamesha **bade data type** mein aata hai (int/float -> float, int/double -> double). Ye sirf division nahi, baaki operations mein bhi hota hai.
- **Type casting kaam aati hai:** `5 / (double)2` = 2.5.
- **Dhyan:** `int answer = 5 / 2.0;` mein right side 2.5 banta hai, par left side **int** hai, to **2** store hota hai.

```
int a = 5;
double b = 2;
cout << a / b;            // 2.5
cout << 5 / 2;            // 2
cout << 5 / (double)2;    // 2.5
```

B. Relational

- **< <= > >= == !=** (== barabar hai?, != barabar nahi hai?). Result hamesha **true (1)** ya **false (0)**.
- **= vs ==:** single = **assign** karta hai, double == **compare** karta hai.

Expression	Result	Print
3 < 5	true	1
3 > 5	false	0
3 <= 3	true	1
3 == 3	true	1
3 == 5	false	0
3 != 5	true	1
3 != 3	false	0

C. Logical

- **|| (OR):** koi bhi ek true ho to true.
- **&& (AND):** sab true ho tabhi true.

- **! (NOT):** answer ko ulta kar deta hai (true -> false, false -> true). Jaise `!(3 > 1)` = 0.
- Inka asli use **conditional statements** (agla chapter) mein samajh aayega.

A	B	A || B	A && B
true	true	true	true
true	false	true	false
false	true	true	false
false	false	false	false

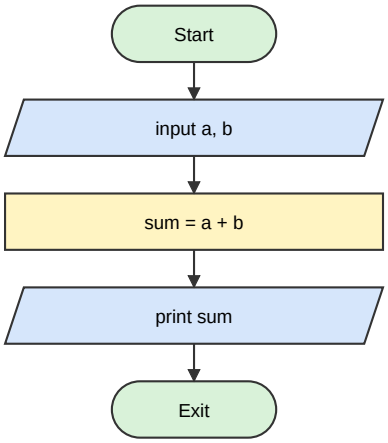
D. Bitwise: binary number system ke baad padhenge.

12. Program: Do numbers ka sum

Sum of 2 numbers

- Structure hamesha: **input -> kaam -> output**.
- Pseudocode ko seedha C++ syntax mein badla (**syntax** = C++ ke rules).

```
input a, b
sum = a + b
print sum
exit
```




```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter a: ";
    cin >> a;
    cout << "Enter b: ";
    cin >> b;
    int sum = a + b;
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Example: a = 3, b = 10 -> Sum = 13.

13. Unary Operators (++ aur --)

- **Binary operator:** do operands chahiye (+, -, *, /). **Unary operator:** sirf **ek** operand chahiye.
- **a++** ka matlab **a = a + 1**. Loops mein baar-baar +1 karna padta hai, isliye shortcut. **--** ka matlab **-1**.
- Dono tarah ke hote hain, farq **kaam aur update ke order** mein hai:

Type	Likhna	Order	a = 10 par b ki value
Post-increment	b = a++	Pehle kaam, baad mein update	b = 10, a = 11
Pre-increment	b = ++a	Pehle update, baad mein kaam	b = 11, a = 11
Post-decrement	b = a--	Pehle kaam, baad mein update	b = 10, a = 9
Pre-decrement	b = --a	Pehle update, baad mein kaam	b = 9, a = 9

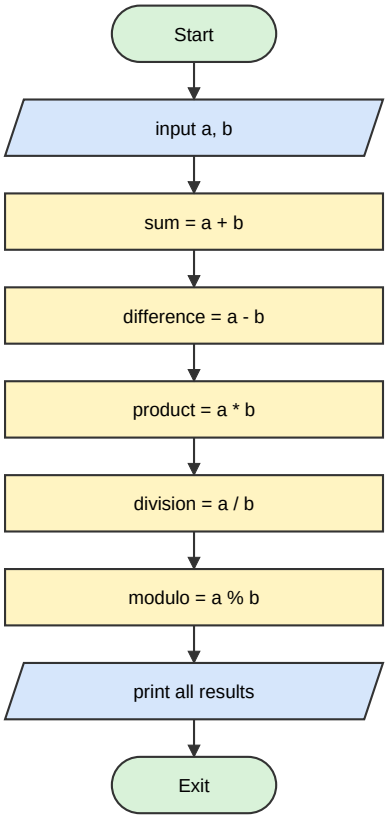
```
int a = 10;
int b = a++;   // b = 10 (pehle kaam), phir a = 11
cout << a << " " << b;   // 11 10
```

- **Post** = baad mein (++ baad mein aata hai), **Pre** = pehle (++ pehle aata hai).
- a ki value to hamesha update hoti hai, farq sirf ye hai ki b mein **purani** value jaati hai ya **nayi**.
- Is topic ka asli use **loops** padhne ke baad clear hoga.

14. Homework

Calculator program

- **Do numbers** ka calculator banao jo **sum, difference, product, division aur modulo** sab kare.
- Sabke liye arithmetic operators already pata hain.
- Pehle flowchart / pseudocode, phir C++ code.



15. Yaad rakhne wali baatein

- C++ **case sensitive** hai, har statement ke end mein **;**.
- Boilerplate: **#include <iostream>**, **using namespace std;**, **int main()**, **return 0;**.
- Code ke 2 stage: **compile** (g++ code.cpp) -> **run** (./a.out).
- **int 4, char 1, float 4, bool 1, double 8 bytes**. float ke baad **f**.
- **char** memory mein ASCII number hota hai (A = 65, a = 97). **bool** print par 1 / 0.
- **int / int = int** (decimal kat jaata hai). Casting: (int)price decimal kaat deti hai.
- **=** assign, **==** compare. Relational operators 1 / 0 dete hain.
- **a++** pehle kaam baad mein update, **++a** pehle update baad mein kaam.
- Har program ka structure: **input -> kaam -> output**.